Sprawozdanie z projektu
Symulator Mikroprocesora 16-bitowego

Tomasz Nazar 197613, Filip Pietrzak 198275

1 kwietnia 2026

1 Sformułowanie zadania i założenia szczegółowe

Zadaniem projektowym było stworzenie programowego modelu symulatora mikroprocesora wzorowanego na architekturze pokrewnej serii Intel 8086. Aplikacja została zrealizowana w języku Python z wykorzystaniem biblioteki GUI PyQt6.

1.1 Założenia architektury i składni

Przyjęto następujące założenia precyzujące działanie wirtualnego procesora:

- **Rejestry:** Symulator posiada 4 główne, 16-bitowe rejestry ogólnego przeznaczenia: AX, BX, CX, DX. Wewnętrznie obsługiwany jest podział każdego z nich na 8-bitowe części starsze (High – AH, BH, CH, DH) oraz młodsze (Low – AL, BL, CL, DL).
- **Lista rozkazów:** Zaimplementowano operacje kopiowania danych (`MOV`), sumowania (`ADD`) oraz odejmowania (`SUB`). Wystąpienie błędu niedomiaru lub nadmiaru (*overflow*) jest automatycznie obcinane maską bitową do szerokości rejestru, zgodnie z działaniem układów sprzętowych (ALU).
- **Składnia (Parsing):** Aplikacja akceptuje klasyczną notację asemblera: `OP DEST, SRC`.
  - Separator argumentów (przecinek) jest traktowany jako opcjonalny.
  - Kod jest automatycznie formatowany do wielkich liter (*case-insensitive*).
  - Tryb natychmiastowy obsługuje systemy dziesiętne i szesnastkowe (poprzedzone prefiksem `0x`).
- **Bezpieczeństwo typów:** Analizator rzuca wyjątek składniowy w przypadku próby przeprowadzenia operacji na operandach o różnej szerokości bitowej (np. `MOV AX, BL` lub próby wczytania liczby 300 do 8-bitowego rejestru `AL`).

2 Opis przyjętych rozwiązań programowych

2.1 Model pamięci i operacje bitowe

Zamiast tworzyć osobne zmienne dla części H i L, przyjęto optymalne podejście sprzętowe. W klasie `CPU` istnieje tylko jeden 16-bitowy licznik dla każdego rejestru. Dostęp do części 8-bitowych realizowany jest w locie za pomocą masek bitowych (*bitwise operations*).

```python
def set_val(self, reg_name, value):
    # ... fragment kodu ...
    if reg_name[1] == 'H':
        # Zachowaj czesc mlodsza (0x00FF), nadpisz starsza
        self.registers[base] = (current & 0x00FF) | ((value & 0xFF) << 8)
    elif reg_name[1] == 'L':
        # Zachowaj czesc starsza (0xFF00), nadpisz mlodsza
        self.registers[base] = (current & 0xFF00) | (value & 0xFF)
```

Listing 1: Zapis wartości do części młodszej (L) i starszej (H)

2.2 Parsing i śledzenie wykonywania programu

Za śledzenie krokowe odpowiada wskaźnik `self.current_line`. Metoda `execute_step()` koloruje aktualnie wykonywaną linię w komponencie `QListWidget`. Analiza składniowa dokonywana jest przez moduł `re` (wyrażenia regularne), który precyzyjnie dzieli kod na mnemotechnik i operandy, weryfikując ich ułożenie.

```python
def parse_and_execute(self, line):
    line = line.split(';')[0].strip().upper() # Usuniecie komentarzy
    # Wyrazenie dopasowujace: np. MOV AX, 0xFF
    match = re.match(r'^([A-Z]+)\s+([A-Z]{2})\s*,?\s*(.+)$', line)

    if not match:
        raise SyntaxError(f"Nieprawidlowa skladnia: {line}")

    op, dest, src = match.groups()
    # ... nastepuje wykonanie operacji zalezne od wyciagnietych grup
```

Listing 2: Parsowanie linii za pomoca wyrazen regularnych

3 Dyskusja osiągniętych wyników (Wady i Zalety)

3.1 Zalety napisanej aplikacji

- **Interaktywność wizualna:** Zaimplementowany interfejs graficzny pozwala na podwójną interakcję. Zmiany stanu wpisane w oknie programu odzwierciedlają się dynamicznie na wirtualnych "diodach"poszczególnych bitów. Z kolei ręczne klikanie diod automatycznie re-kalkuluje wartość szesnastkową w rejestrze, co wyśmienicie sprawdza się w celach dydaktycznych.

- **Odporność na błędy programisty:** Zastosowana maszyna stanów w parsingu uniemożliwia zawieszenie programu. Wszystkie błędy asemblerowe (zła nazwa rejestru, zła wielkość literału, przekroczenie 16-bitów) są wychwytywane i zgłaszane jako okna z ostrzeżeniami ze wskazaniem konkretnej linii kodu.

3.2 Wady i ograniczenia

- **Ograniczona pula rozkazów:** Symulator obsługuje jedynie instrukcje przesłania i prostej arytmetyki. Brak zaimplementowanych instrukcji sterujących (skoków JMP, pętli) sprawia, że wykonywane programy mają wyłącznie charakter sekwencyjny.

- **Brak Rejestru Flag (FLAGS):** Wykonanie instrukcji arytmetycznych w symulatorze nie ustawia odpowiednich flag sprzętowych (Carry Flag, Zero Flag, Overflow Flag). W prawdziwym mikroprocesorze rejestr ten jest absolutnie kluczowy do kontroli wyników operacji i podejmowania decyzji skoków warunkowych.
- **Brak pamięci RAM:** Program obsługuje adresowanie rejestrowe i natychmiastowe, ale pomija adresowanie pośrednie i bezpośrednie z użyciem komórek pamięci (np. `MOV AX, [1000h]`).

Podsumowując, napisana aplikacja osiąga 100% zgodności z postawionym zadaniem. Posiada świetnie zaprogramowany interfejs weryfikujący oraz zaawansowaną logikę dzielenia i łączenia bitów między fragmentami H i L rejestrów, stanowiąc solidną bazę (szkielet) pod budowę emulatora operującego na pełnej architekturze pamięciowej.